Computing for Analytical Work

Session 2 — What is running?

Block 4 — Notebooks, scripts, state, and reproducibility

This block in one sentence

A notebook is an interface to a running Python process with hidden state. A script starts fresh each time.

Hidden state is why a notebook can look finished and be broken.

What we cover

- Cells, the kernel, and what the kernel remembers
- Out-of-order execution and variables from earlier runs
- Imports after first use; outputs from a previous run
- Restart Kernel and Run All — the execution test, and what it does not test
- Notebook vs script; minimal reproducibility
- Where the kernel comes from: `uv run jupyter lab`, again
- Git thread: commit messages and `.gitignore`
- AI thread, stretch, Lab 4, and Homework 2

What a notebook actually is

- The `.ipynb` file: cells of code, plus **saved outputs** from whenever someone last ran them
- The **kernel**: a live Python interpreter, started when you open the notebook
- The page you see is a *transcript*. The kernel is the *process*.

You edit the transcript. The kernel remembers only what you actually ran, in the order you ran it.

Cell execution and kernel state

```
# cell 1
df = pd.read_csv("data/raw/sales.csv")
```

```
# cell 2
total = df["amount"].sum()
```

- Run cell 1: the kernel now holds `df`. Run cell 2: it holds `total` too.
- Delete cell 1 from the page. `df` is **still in the kernel.** Cell 2 still works.
- The `[3]` next to a cell is the *order it ran in*, not its position on the page.

Out-of-order execution

- You write cell 5, run it, see a bug, fix cell 2, re-run **only cell 2**, then re-run cell 5
- Cells 3 and 4 never saw the new cell 2. Their results are from the old one.
- The page reads top to bottom. The kernel did not run top to bottom.

Every notebook you edit for more than ten minutes is in this state. It is normal. It is also why nothing on the page is evidence until you run it clean.

Variables that exist only because of earlier runs

- You renamed `sales` to `df` in cell 2. Cell 7 still says `sales`. It still works.
- Because `sales` is still in the kernel, from before the rename.
- Save, close, reopen tomorrow, run cell 7: `NameError: name 'sales' is not defined`

The notebook did not break overnight. It was broken this afternoon. The kernel was covering for it.

Imports after first use

```
# cell 4
import pandas as pd
```

- `pd` was first used in cell 1. It worked, because you ran cell 4 last week.
- Fresh kernel, top to bottom: cell 1 fails with `NameError: name 'pd' is not defined`
- Rule: **all imports in the first cell.** Then a fresh kernel cannot lie about them.

The "output exists only from a previous run" trap

- The saved `.ipynb` shows a beautiful chart under cell 9
- That chart was rendered on someone's kernel, some day, in some state
- You open it, look at the chart, and think "this works." **You have not run anything.**
- Saved outputs are a screenshot of the past. They are not evidence that the code runs.

Same trap with `output/` folders: a file exists, so the step "worked." Did it, today, from this code?

The truth test

Restart Kernel and Run All

Kernel menu → **Restart Kernel and Run All Cells**

- Kills the process. Every variable is gone. Every import is gone.
- Runs every cell, top to bottom, in page order, from nothing.
- If it finishes clean, every cell runs in page order from nothing. If it fails, the page was lying.
- Clean says nothing about whether the numbers are right. That is a second, separate check against the data.

"It worked when I ran it" is not evidence. **Restart and Run All is the required execution test.** Before you commit. Before you submit. Before you say "done."

Notebook vs script

- **Notebook** — exploring. Look at a dataframe, try a plot, change your mind. State is the feature.
- **Script** — repeating. Same input, same steps, same output, every time. No state survives.
- A script cannot carry kernel state between runs; it starts from nothing every time. Its *inputs* can still change its output. For this course's scripts: same input, same output.

When a notebook has stopped changing and you just need it to run again tomorrow, that is a script.

Minimal reproducibility

The only question: **can I restart and run it again?**

- Fresh kernel, or a fresh `uv run python scripts/analyze.py`, on a fresh `uv sync`
- Same output as last time, from the code as it is on the page, right now
- Not "did it ever work." Not "does the chart look right." *Will it work again, from nothing?*

This is the standard for every deliverable in this course from today on.

Where the kernel comes from — the rule, again

Open the **project folder** in VS Code → kernel picker → the `.venv/` interpreter.
(Or `uv run jupyter lab` from the folder; verify the same way.)

- A global kernel, Anaconda, or another project's `.venv` : **right code, wrong Python**
- The cells are correct. The kernel searches a different `sys.path` . `ModuleNotFoundError` .
- First check in any confused notebook: `import sys; print(sys.executable)` in a cell

A whiff of Git — what not to commit

Good commit messages

```
git commit -m "Move imports to first cell; notebook passes Restart and Run All"
```

- Say **what changed and why it is now correct**. Present tense. One line.
- A message is for the person reading `git log` in six weeks. That person is you.
- Bad: `fix`, `update`, `final`, `final2`, `asdf`, `changes`

The message should let a reader decide whether to look at the diff without looking at the diff.

Commit early, commit ugly

- Your history is **for future-you, not an audience**. Nobody is grading your prose at 11pm.
- Five ugly checkpoints beat one polished commit at the end — because checkpoints are what `git restore` can reach when you break something
- Committing once at the end defeats the point: it protects nothing along the way
- One thing to *recognize*, not learn: `git status` starts with `On branch main`. A branch name you don't recognize there? **Stop and ask a TA.** Branches are a later course.

`.gitignore` — what it is

- A plain text file at the repo root, listing paths Git should **skip**
- Ignored paths are skipped by `git status` and by `git add .`, so the ordinary accident cannot happen
- Two limits: a file Git already tracks stays tracked, and `git add -f` forces one in. A guard, not a law.
- Every course repo ships the same standard one. You will read it, not write it.

Git is for source work and meaningful checkpoints, not local junk. The next four slides walk the file, section by section.

`.gitignore` — Python caches and virtual environments

```
# Python
__pycache__/
*.py[cod]

# Virtual environments
.venv/
venv/
```

- `__pycache__/` and `*.pyc` — compiled bytecode Python regenerates on every run. Pure clutter.
- `.venv/` — the environment. Hundreds of megabytes, machine-specific, **rebuilt by `uv sync`** from `pyproject.toml` and `uv.lock`. Never commit it.

`.gitignore` — Jupyter checkpoints and editor junk

```
# Jupyter
.ipynb_checkpoints/
*-checkpoint.ipynb

# IDE / editor
.vscode/
.idea/
.DS_Store
Thumbs.db
```

- Checkpoints are Jupyter's autosave copies. Stale duplicates of your notebook, one per folder.
- `.DS_Store`, `Thumbs.db`, `.vscode/` — your OS and editor's private notes. Not the project.

`.gitignore` — generated output, logs, and secrets

```
# Course-specific: generated output
output/
*.log

# Secrets — never commit these
.env
*.key
credentials.json
```

- `output/` — anything the code *produces* is not source. Rerun the code to get it back.
- Secrets: a committed key is public forever, even after you delete it. History remembers.

What is deliberately *not* ignored: `data/`

- There is no `data/` line. That is on purpose.
- Course data files are small and are **committed with the code**
- Clone the repo, and the data is right there. The labs are about loading and inspecting it.
- Do not add a `data/` line "just in case." If a future lab has a large file, that lab's `.gitignore` says so.

Rule of thumb: source, small data, `pyproject.toml`, `uv.lock` — in. Environments, caches, outputs, secrets — out.

AI thread — three good uses today

- *"Here is my notebook. Which cells depend on state from cells that run later? Where are the hidden-state risks?"*
- *"Restart and Run All fails at cell 3 with this* `NameError`*. Here is the cell order and the traceback. Why?"*
- *"These four cells repeat the same cleaning steps. Help me turn them into one function."*

Give it the actual cell order and the actual traceback. Then restart and run it yourself.

AI can clean up code. Restart and Run All is the truth test.

- The assistant can reorder cells, name things better, and write the function. Good.
- It cannot run your kernel. It does not know what your kernel remembers.
- Every AI suggestion in a notebook ends the same way: **Restart Kernel and Run All.** Clean or not clean.

That is the third time. It will be on the exam.

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five.

Lab 4 — The notebook lies

```
git clone <the Lab 4 starter>
cd <the folder it created>
uv sync
```

- Open the **folder** in VS Code; open `notebooks/clean_and_plot.ipynb` ; kernel → `.venv/`
- The saved outputs look correct. Every result is there. **Do not trust them.**
- **Restart Kernel and Run All Cells** → it fails with `NameError`
- Cells depend on cells that come *later* on the page. Find them. Fix the order, or rewrite.
- Then, with a clean tree after your commit: delete a cell, save, `git status` , `git restore` the notebook, run clean again. Five minutes. Homework 2 grades this move.

What you produce, and the checkoff

1. The notebook passes **Restart Kernel and Run All Cells** — clean, top to bottom, twice
2. Imports in the first cell; no cell depends on a later one
3. One commit with a message that says what you fixed; `git status` clean afterward, no `.ipynb_checkpoints/` or `.venv/` in it
4. Recovery practised: a deleted cell brought back with `git restore`, then a clean run again
5. `DIAGNOSIS.md`: symptom, cause, evidence (the `NameError` line, the cell numbers), change, verification
6. Checkoff: explain it to a TA in about a minute. No notes. Then one what-if question about kernel state.

Stretch: repeated cleaning steps into one function; a script that runs the core workflow fresh; one sanity check. Resets in the README; both destroy work.

Homework 2 assigned

Python execution, environments, notebooks, and Git checkpoints — 15% of the grade, 6–7 hours. Due the **Friday after this session, 23:59** — slot on Moodle.

A repo that fails for environment reasons *and* notebook execution-order reasons. Deliver:

- working setup instructions
- dependency file (`pyproject.toml` + `uv.lock` , via `uv add`)
- a notebook that passes Restart Kernel and Run All
- **two** meaningful commits — one after the environment fix, one after the notebook fix — with real messages
- a diagnosis note per failure, with the **reconciliation**: script table = notebook numbers, to the cent; selection = the README's threshold
- `RECOVERY.md` : break a committed file on purpose, `git restore` it, prove it
- a **60–90 second video**: Restart and Run All succeeding, then what broke, what changed, how you verified